Software Impacts 26 (2025) 100794



Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts



Original software publication

MDFlaker: A tool for multi-factor detection and root cause analysis of flaky tests



Azeem Ahmad

Prince Sultan University, Kingdom of Saudi Arabia

ARTICLE INFO

Keywords:
Flaky tests
Non-deterministic tests
Flaky test detection
Automated root-cause analysis
Trace-back

ABSTRACT

Test flakiness, characterized by inconsistent test outcomes without changes to the codebase, is a prevalent issue in continuous integration (CI) environments. This paper introduces a practical tool named *MDFlaker* that implements a multi-factor approach for detecting flaky tests, grounded in empirical research. The tool analyzes four key factors—traceback coverage, test smells, flakiness frequency, and test size—and applies a K-Nearest Neighbors (KNN) model for classification. Depending on the test case's execution history, the tool switches between traceback-prioritized or multi-factor classification strategy. Integrated with platforms like GitHub and Travis CI, it processes real-time data from commits, builds, and test executions to support both research experimentation and industrial deployment. By improving the accuracy and interpretability of flaky test detection, the tool helps developers regain trust in CI pipelines, reduces debugging overhead, and fosters high-quality, rapid software delivery. This work not only enhances software reliability in practice but also opens new avenues for research in automated software quality assurance.

Code metadata

Current code version	*For example: v1.0*
Permanent link to code/repository used for this code version	https://github.com/SoftwareImpacts/SIMPAC-2025-241
Permanent link to Reproducible Capsule	
Legal Code License	*Apache License*
Code versioning system used	*GIT*
Software code languages, tools, and services used	*Python*
Compilation requirements, operating environments & dependencies	
If available Link to developer documentation/manual	https://github.com/azeem599/SIMPAC_FT
Support email for questions	aahmad@psu.eda.sa

1. Introduction

Rapid feedback from CI/CD testing pipelines is essential, but equally important is the quality and reliability of that feedback [1–3]. Developers' productivity is negatively affected when test results are not only delayed but also unreliable or inconsistent—a phenomenon known as *test flakiness* [4–8]. Ideally, developers expect test failures to be a direct consequence of their recent code changes. However, in practice, many failures arise from flaky tests—those that intermittently pass or fail without any changes to the underlying codebase [9]. A widely accepted definition of a flaky test is one that produces both passing and failing results under identical conditions [10]. These non-deterministic outcomes erode developers' trust in the CI system and slow down the development cycle. Flaky tests are a prevalent issue in large-scale software systems. For instance, Google has reported that approximately 80% of test failures are attributed to flakiness, with only 20% linked to actual code regressions [11]. Another internal study at Google found that 1 in 7 tests could fail due to flakiness [12].

A number of recent contributions are gaining attention in the quickly developing field of flaky test detection. Tools like iDFlakies [5] and DeFlaker [7] have investigated various detection techniques, such as heuristic-based evaluation and dynamic analysis. But the majority of these technologies either only employ one kind of evidence or do not facilitate useful root cause analysis. We present **MDFlaker**, a useful

E-mail address: aahmad@psu.edu.sa.

https://doi.org/10.1016/j.simpa.2025.100794
Received 20 August 2025; Received in revised form 22 October 2025; Accepted 28 October 2025

2665-9638/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

and research-based tool [13] for flaky test identification and root cause analysis

Our prior research, titled *"A Multi-factor Approach for Flaky Test Detection and Automated Root Cause Analysis"* [13], introduced the theoretical foundation for identifying flaky tests using a combination of empirically validated factors. As part of that study, we developed a prototype tool called *MDFlaker* to support data collection and analysis from open-source projects. This tool enabled automated extraction of flakiness related features such as test smells, historical pass/fail transitions, traceback coverage, and test size as well as facilitated the training and evaluation of a KNN-based classifier.

In this paper, we present the **matured and extended version of MDFlaker** as a reusable and standalone software artifact. It has been refined beyond the scope of the original study to enable broader applicability and practical usage by other researchers and practitioners. The tool now supports integration into CI pipelines, improved visualization of flaky patterns, and customizable configurations for detection thresholds making it suitable for industrial deployment and further empirical studies.

2. Tool architecture and working mechanism

Fig. 1 presents the architecture of our tool, which detects flaky tests by analyzing software artifacts collected from GitHub repositories and Travis CI. The pipeline is divided into three main phases: input collection, processing, and classification.

2.1. Input collection

The tool collects three sources of input:

- **Code snapshots** from GitHub.
- **Commit-level changes** in JSON format.
- **Build history and logs** from Travis CI, including pass/fail outcomes.

These inputs are processed and parsed using Python scripts and log extractors, and passed into the next phase for feature extraction.

2.2. Processing and feature extraction

The tool extracts the following four core features, which are used as inputs to a *k-Nearest Neighbors (KNN)* classifier for flaky test prediction:

1. **Traceback Coverage:** Whether the traceback of the test failure covers recent code changes as suggested by Bell et al. [7]
2. **Test Smells:** Detection of flakiness-inducing patterns such as sleep, randomness, or improper isolation.
3. **Flakiness Frequency:** Calculated as the number of pass/fail transitions divided by the total number of failures for a test case.
4. **Test Size:** Measured as the number of uncommented lines of code in the test method.

All extracted features are stored in a database, which also keeps historical state and results for traceability and incremental analysis.

2.3. Classification strategy

The tool applies different decision rules based on the historical state of each test case:

Scenario 1: Traceback takes priority. If a test case failed but had previously passed, the tool prioritizes the **traceback coverage** factor. If the traceback does not cover the recent changes, the failure is labeled as flaky. Otherwise, the test is considered non-flaky. This targeted approach helps reduce noise from legitimate test failures.

Scenario 2: Multi-factor detection. If the failed test did not previously pass (i.e., the state was Failed, Error, Canceled, or None), then all four features are considered equally using a multi-factor classification approach. This comprehensive strategy enables deeper detection for persistent or systemic flakiness.

2.4. Output and feedback

After classification, the results are stored and visualized. Developers receive insights including the final flaky/non-flaky label, associated feature values, and the prioritized decision path (e.g., whether traceback alone or all factors were used).

This approach ensures both high recall and interpretability, addressing key challenges in test flakiness detection and enabling more reliable continuous integration practices.

3. Tool output and developer benefits

The tool provides a variety of visual insights that help software engineers detect flaky tests and diagnose their root causes effectively. By analyzing test case characteristics, historical build outcomes, and smell patterns, the tool surfaces multiple indicators that are difficult to spot manually. The following figure illustrates key visualizations generated by the tool.

3.1. Supporting flaky test detection

Fig. 2(a) presents the flaky test detection results across two classifiers: test case-only features (TC) and multi-factor features (MF). These outcomes enable practitioners to directly identify potentially flaky test cases and compare classification strategies. The separation between `TC_flaky` and `MF_flaky` allows teams to assess the impact of additional root cause features. Fig. 2(b) provides an overview of historical build success and failure rates, which is useful to identify unstable projects or configurations.

Fig. 4 demonstrates the proportion of test cases with and without smells, the number of smells per test case, and the distribution of specific smell types (e.g., randomness, network, async wait). These insights allow developers to pinpoint potential causes of flakiness, such as reliance on external resources or non-deterministic behaviors.

In addition, re-running test cases can indicate which are flaky, but it does not provide sufficient or no evidence about the root causes of test flakiness. As part of the reporting, MDFlaker provides different types of information (e.g., CSV files or graphs) about either each test case or a complete test suite that can help them resolve test flakiness or at least provide an indication about where to start looking. Fig. 3 presents an image generated by MDFlaker. This image reveals the proportion of test smells in a project under investigation

3.2. Impact on root cause analysis

Combined, these outputs provide both a high-level summary and drill-down capability for developers and testers. The integration of size, history, smell, and classifier output allows users to explore multi-dimensional patterns, ultimately improving the robustness and reliability of automated test suites.

4. Impact overview

The modular architecture of MDFlaker also makes it adaptable to learning-based systems, where test outcomes can influence retraining or deployment decisions. By providing reliable, multi-factor flaky test detection, our tool supports trustable data collection in AI-driven CI/CD workflows, addressing a growing need for dependable test infrastructure in ML engineering. We discuss in detail the research and practical impacts in the following sections.

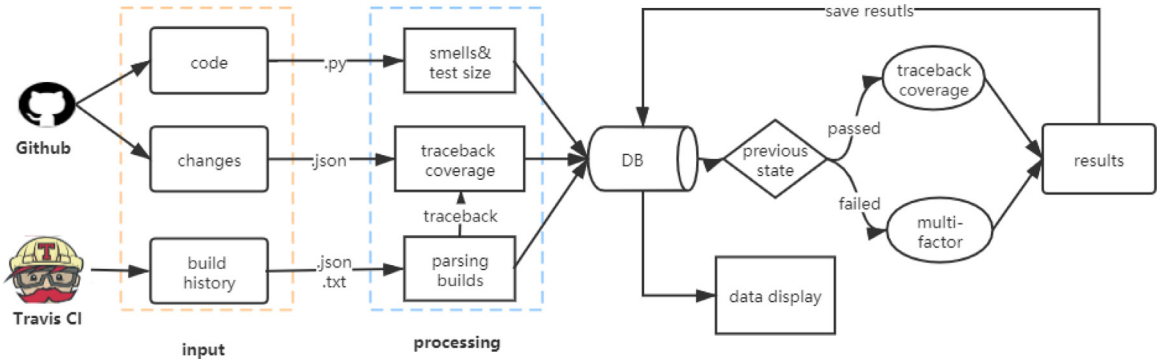
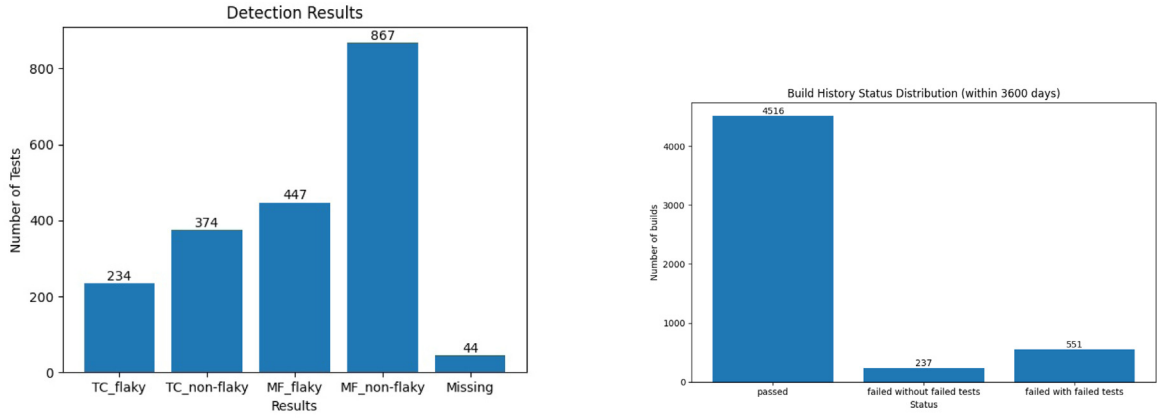


Fig. 1. Tool architecture and event flow through CI/CD systems into MDFlaker.



(a) Detection Results Distribution (b) Build History Status Distribution

Fig. 2. Key Visual Outputs from the Tool.

Test Case	Number of Smells	Smell type	Tip	Location	Path
test_lex_attrs_l	6	Network	URL	58	D:\spacy\tests\lang\test_attrs.py
test_lex_attrs_l	6	Network	URL	58	D:\spacy\tests\lang\test_attrs.py
.....	.....	.....	.....	.....	.....
test_issue1506	5	Training	For in range	24	D:spacy\tests\regression\test_issue1501-2000.py
test_issue1506	5	Training	For in range	26	D:\spacy\tests\regression\test_issue1501-2000.py

Fig. 3. Images generated by MDFlaker to present different types of information about test smells in test cases.

4.1. Research impact

This tool is among the first to use conditional classification based on test execution history since it incorporates past build states and commit-level data, which presents a unique perspective. It also gives researchers an extendable and reusable framework to test machine learning models, test smell measurements, and new flaky detection theories. Additionally, as mentioned in recent literature, our work verifies and adds to previous research on the occurrence and difficulties of flaky tests in large-scale industrial systems.

4.2. Practical impact

This solution solves a significant issue in continuous integration (CI) systems from an industrial standpoint: confidence in test results. Non-deterministic test failures are a common problem for developers, slowing down delivery pipelines and decreasing developer productivity. More dependable feedback during CI/CD cycles is made possible by this tool, which automatically classifies flaky tests using interpretable criteria with little developer's intervention.

The solution is simple to include into current DevOps workflows and works with well-known platforms like Travis CI and GitHub. By employing the tool, teams can boost trust in test feedback, decrease the amount of time spent debugging flaky failures, and eventually improve product quality and deployment speed. As flaky test detection becomes a major concern in software engineering methods, the tool has the potential to be extensively implemented and has already demonstrated encouraging results in practical experiments.

The MDFlaker presented in this paper is the outcome of our earlier research described in Ahmad et al. (A Multi factor Approach for Flaky Test Detection and Automated Root Cause Analysis) [13].

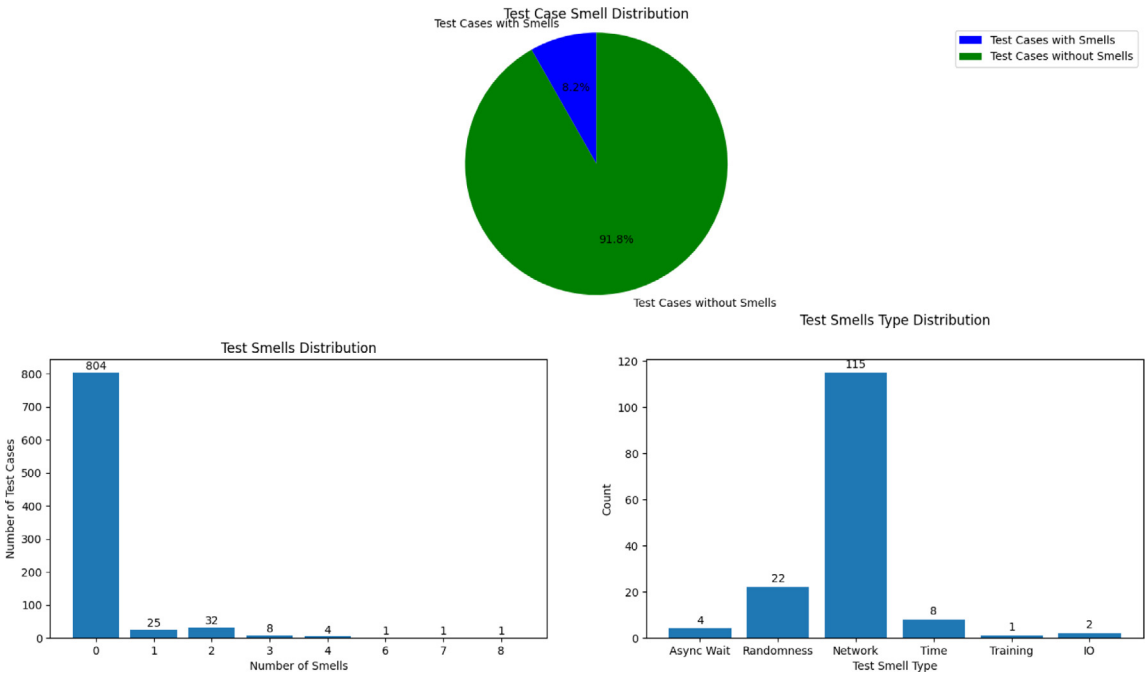


Fig. 4. Test Smell and Type Distributions.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge the support provided by the Prince Sultan University, Saudi Arabia, which covered the Article Processing Charges (APC) for this publication. The university's management commitment to advancing research and knowledge dissemination has been invaluable in successfully completing this research. I also thank Linkoping University as this work was conducted at Linkoping University.

References

[1] O. McHugh, K. Conboy, M. Lang, Agile practices: The impact on trust in software project teams, IEEE Softw. 29 (3) (2012) 71–76.
[2] A. Ahmad, X. Sun, M.R. Naeem, Y. Javed, M. Akour, K. Sandahl, Understanding flaky tests through linguistic diversity: A cross-language and comparative machine learning study, IEEE Access (2025).
[3] A. Ahmad, E.N. Held, O. Leifler, K. Sandahl, Identifying randomness related flaky tests through divergence and execution tracing, in: Proceedings of the 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW, IEEE, 2022, pp. 293–300.
[4] Q. Luo, F. Hariri, L. Eloussi, D. Marinov, An empirical analysis of flaky tests, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2014, pp. 643–653.
[5] W. Lam, R. Oei, A. Shi, D. Marinov, T. Xie, Idflakies: A framework for detecting and partially classifying flaky tests, in: 12th IEEE Conference on Software Testing, Validation and Verification, ICST, 2019, pp. 312–322.
[6] A. Ahmad, Contributions To Improving Feedback and Trust in Automated Testing and Continuous Integration and Delivery, Linköping University, Sweden, 2022.
[7] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, D. Marinov, Deflaker: Automatically detecting flaky tests, in: 2018 IEEE/ACM 40th International Conference on Software Engineering, ICSE, pp. 433–444.
[8] A. Ahmad, O. Leifler, K. Sandahl, Empirical analysis of practitioners' perceptions of test flakiness factors, Softw. Test. Verif. Reliab. 31 (8) (2021) e1791.
[9] A. Ahmad, An evaluation of machine learning methods for predicting flaky tests, in: Proceedings of the 8th International Workshop on Quantitative Approaches To Software Quality, Vol. 2767, QuASoQ 2020, 2020, pp. 37–46.
[10] M. Fowler, Eradicating Non-Determinism in Tests, [Online]. Available: https://martinfowler.com/articles/nonDeterminism.html.
[11] C. Leong, A. Singh, M. Papadakis, Y.L. Traon, J. Micco, Assessing transition-based test selection algorithms at google, in: Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice, 2019, pp. 101–110.
[12] J. Micco, Flaky Tests at Google and How We Mitigate Them, [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html.
[13] Azeem Ahmad, Kristian Sandahl, Ola Leifler, A multi-factor approach for flaky test detection and automated root cause analysis, in: Proceedings of the 2022 IEEE/ACM International Conference on Software Engineering, 2022.